

16. 上位 MPC 目的関数

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	6
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 デコレータと @dataclass	7
6.9 制御、状態、入力、モデル予測制御 MPC	8
6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明	8
6.11 車両力学、電力収支、効率 map	8
7 関数・クラスを上から順に解説	9
7.1 L8 クラス UpperCostConfig	9
7.2 L44 関数 UpperCostConfig.to_dict	10

7.3	L48 関数 <code>_cfg_value</code>	11
7.4	L54 関数 <code>load_upper_cost_config</code>	12
7.5	L65 関数 <code>load_upper_cost_config.pick</code>	14
7.6	L110 関数 <code>quad_penalty</code>	14
7.7	L118 関数 <code>upper_stage_cost</code>	15
7.8	L263 関数 <code>upper_terminal_cost</code>	19
7.9	L283 関数 <code>active_upper_cost_terms</code>	20
8	処理の流れ	21
9	このファイルの位置づけ	21

1 対象

項目	内容
ファイル	mpc_solarcar/upper_cost.py
ソース SHA-256	b69065249298ad089d085bcb06d9277c10dae6b61ec0d334d69a13ffef56a3bb
種別	Python
区分	planner helper
役割	速度、SoC、温度、電流、進捗、day-end、terminal 条件を penalty として定義する。
起動文脈	upper planner の良し悪しを数式化する場所。

2 このファイルを読む理由

速度、SoC、温度、電流、進捗、day-end、terminal 条件を penalty として定義する。

- upper_stage_cost と upper_terminal_cost が中心。
- load_upper_cost_config が profile の重み束を解く。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py
- scripts/solar_sim.py
- tune_upper_planner_weights.py

3.2 次に読むと理解が進むファイル

- scripts/tune_upper_planner_weights.py

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
@dataclass
class UpperCostConfig:
    objective_mode: str = "weighted"
    w_wait: float = 1.0
    w_travel_time: float = 1.0
    w_terminal_soc_min: float = 30.0
    w_day_end_soc_min: float = 1.0e5
```

```

w_soc_day_max: float = 1.0e4
w_soc_day_track: float = 0.0
w_speed_smooth: float = 30.0
w_dv_limit: float = 2.0
w_speed_limit: float = 50.0
w_drive_window: float = 1.0e5
w_current_sq: float = 0.01
w_pack_energy: float = 0.0
w_joule_loss: float = 0.0
w_aero_energy: float = 0.0
w_mech_energy: float = 0.0
w_speed_quartic: float = 0.0
w_solar_headroom: float = 0.0

```

5 主要構造の読み取り

主要クラスは `UpperCostConfig`。主要関数は `to_dict`, `load_upper_cost_config`, `pick`, `quad_penalty`, `upper_stage_cost`, `upper_terminal_cost`, `active_upper_cost_terms`。

5.1 主要クラス・関数

行	種別	名前	補足
8	class	<code>UpperCostConfig</code>	
44	function	<code>to_dict</code>	args: self
48	function	<code>_cfg_value</code>	args: cfg, key, default
54	function	<code>load_upper_cost_config</code>	args: mpc_cfg
65	function	<code>pick</code>	args: name, default
110	function	<code>quad_penalty</code>	args: x, cap
118	function	<code>upper_stage_cost</code>	args: cfg
263	function	<code>upper_terminal_cost</code>	args: cfg
283	function	<code>active_upper_cost_terms</code>	args: cfg, threshold

5.2 ファイルを上から読んだときの定義順

行	内容
8	クラス <code>UpperCostConfig</code> を定義する。
48	関数 <code>_cfg_value</code> を定義する。
54	関数 <code>load_upper_cost_config</code> を定義する。
110	関数 <code>quad_penalty</code> を定義する。
118	関数 <code>upper_stage_cost</code> を定義する。
263	関数 <code>upper_terminal_cost</code> を定義する。
283	関数 <code>active_upper_cost_terms</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>from dataclasses import asdict, dataclass</code>	dataclasses から asdict, dataclass を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L7, L45。
4	<code>from typing import Any, Dict, Optional</code>	typing から Any, Dict, Optional を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L44, L48, L55, L57, L126, L132, L145, L148, ...。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Python インタプリタが読み込む -> OS 上のプロセス -> Python オブジェクトをメモリに生成 -> 関数や callback を実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# node と alias は同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):  
    return min(maximum, max(minimum, value))  
  
v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが‘self.z’を更新すればオブジェクトの状態が残るが、単に‘z’を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の‘console_scripts’は、端末で使う実行可能名と‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->  
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’ へ渡す。‘self’ は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self.step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 デコレータと @dataclass

‘@名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を書く。

‘@dataclass’は型注釈付きフィールドから ‘__init__’、‘__repr__’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は ‘__post_init__’ や別の検証関数で確認する必要がある。

根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

6.9 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

6.10 目的関数、制約、L-BFGS-B、SHGO、有限 grid 証明

数値最適化器は、利用者が与えた目的関数を複数の候補点で評価し、より小さい値を持つ候補を探す。solver が物理を理解するのではなく、物理と運用価値は **cost** 関数へ書かれる。

L-BFGS-B は変数ごとの上下限を扱える局所最適化法である。初期値の近くの谷へ収束し得るため、非凸問題では複数 **seed** や大域探索と組み合わせる。**success** が **False** でも有限な候補が返る場合があるため、採用条件をコード側で決める。

SHGO は定めた **sampling** と局所最適化を組み合わせる大域最適化法である。有限 **Cartesian grid** の全列挙は、その **grid** 上の最良を証明できるが、連続領域全体の最良を自動的に証明しない。資料ではこの証明範囲を区別する。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)
- [SciPy 公式: scipy.optimize.shgo](#)

6.11 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 **power** は概ね ‘ $P_{\text{mech}} = F_{\text{total}} * v + P_{\text{inertia}}$ ’ で、駆動時と回生時で異なる効率 **map** を通して DC 側 **power** へ変換する。**map** は速度・torque などを軸にした実測または同定 **table** であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

7 関数・クラスを上から順に解説

7.1 L8 クラス UpperCostConfig

- 行範囲: L8–L45
- 所属: module 直下
- 定義: UpperCostConfig(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- @ で始まる行は、定義したクラスを加工する decorator である。

上から順の処理

1. objective_mode に 'weighted' を代入する。
2. w_wait に 1.0 を代入する。
3. w_travel_time に 1.0 を代入する。
4. w_terminal_soc_min に 30.0 を代入する。
5. w_day_end_soc_min に 100000.0 を代入する。
6. w_soc_day_max に 10000.0 を代入する。
7. w_soc_day_track に 0.0 を代入する。
8. w_speed_smooth に 30.0 を代入する。
9. w_dv_limit に 2.0 を代入する。
10. w_speed_limit に 50.0 を代入する。
11. w_drive_window に 100000.0 を代入する。
12. w_current_sq に 0.01 を代入する。

代表コード断片

```
class UpperCostConfig:
    objective_mode: str = "weighted"
    w_wait: float = 1.0
    w_travel_time: float = 1.0
    w_terminal_soc_min: float = 30.0
    w_day_end_soc_min: float = 1.0e5
    w_soc_day_max: float = 1.0e4
    w_soc_day_track: float = 0.0
    w_speed_smooth: float = 30.0
    w_dv_limit: float = 2.0
    w_speed_limit: float = 50.0
    w_drive_window: float = 1.0e5
    w_current_sq: float = 0.01
    w_pack_energy: float = 0.0
    w_joule_loss: float = 0.0
    w_aero_energy: float = 0.0
    w_mech_energy: float = 0.0
    w_speed_quartic: float = 0.0
    w_solar_headroom: float = 0.0
    w_progress_lag: float = 0.0
    w_progress_terminal_lag: float = 0.0
    w_kinetic_pos: float = 0.0
    w_pack_power_slew: float = 0.0
    w_temp: float = 5.0
    w_soc_terminal: float = 0.0
    w_soc_floor_barrier: float = 0.0
    w_uncertainty_reserve: float = 0.0
    speed_quartic_scale_kmh: float = 80.0
    progress_lag_deadband_kmh: float = 0.0
    soc_solar_headroom_max: float = 0.92
    solar_headroom_power_scale_w: float = 1000.0
    soc_floor_barrier_eps: float = 0.01
    reserve_soc_per_hour: float = 0.0
    reserve_soc_max_extra: float = 0.0
    constraint_penalty: float = 1.0e4
    ...
```

7.2 L44 関数 UpperCostConfig.to_dict

- 行範囲: L44–L45
- 所属: UpperCostConfig
- 定義: to_dict(self)->Dict[str,Any]
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: asdict
- 戻り値の要点: asdict(self)
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `asdict(self)` を返す。

代表コード断片

```
def to_dict(self) -> Dict[str, Any]:  
    return asdict(self)
```

7.3 L48 関数 `_cfg_value`

- 行範囲: L48–L51
- 所属: `module` 直下
- 定義: `_cfg_value(cfg: Optional[dict], key: str, default)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `isinstance`
- 戻り値の要点: `default / cfg[key]`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `isinstance(cfg, dict) and key in cfg` を判定し、真なら内部処理を行う。
2. `cfg[key]` を返す。
3. `default` を返す。

代表コード断片

```
def _cfg_value(cfg: Optional[dict], key: str, default):  
    if isinstance(cfg, dict) and key in cfg:  
        return cfg[key]  
    return default
```

7.4 L54 関数 load_upper_cost_config

- 行範囲: L54–L107
- 所属: module 直下
- 定義: `load_upper_cost_config(mpc_cfg:Optional[dict],*, legacy:Optional[dict]=None, default_drive_window:float=100000.0)->UpperCostConfig`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `UpperCostConfig, _cfg_value, float, get, isinstance, lower, pick, str, strip`
- 戻り値の要点: `UpperCostConfig(objective_mode=str(pick('objective_mode', 'weighted')).strip().lower(), w_wait=float(pick('w_wait', 1.0)), w_travel_time=float(pick('w_travel_time', 1.0)), w_terminal_soc_min=float(pick('w_terminal_soc_min', 30.0)), w_day_end_soc_min=float(pick('w_day_end_soc_min', default_drive_window)), w_soc_day_max=float(pick('w_soc_day_max', 10000.0)), w_soc_day_track=float(pick('w_soc_day_track', 0.0)), w_speed_smooth=float(pick('w_speed_smooth', _cfg_value(legacy, 'w_dv', 30.0))), w_dv_limit=float(pick('w_dv_limit', 2.0)), w_speed_limit=float(pick('w_speed_limit', 50.0)), w_drive_window=float(pick('w_drive_window', default_drive_window)), w_current_sq=float(pick('w_current_sq', _cfg_value(legacy, 'w_current', 0.01))), w_pack_energy=float(pick('w_pack_energy', 0.0)), w_joule_loss=float(pick('w_joule_loss', 0.0)), w_aero_energy=float(pick('w_aero_energy', 0.0)), w_mech_energy=float(pick('w_mech_energy', 0.0)), w_speed_quartic=float(pick('w_speed_quartic', 0.0)), w_solar_headroom=float(pick('w_solar_headroom', 0.0)), w_progress_lag=float(pick('w_progress_lag', 0.0)), w_progress_terminal_lag=float(pick('w_progress_terminal_lag', 0.0)), w_kinetic_pos=float(pick('w_kinetic_pos', 0.0)), w_pack_power_slew=float(pick('w_pack_power_slew', 0.0)), w_temp=float(pick('w_temp', _cfg_value(legacy, 'w_T', 5.0))), w_soc_terminal=float(pick('w_soc_terminal', 0.0)), w_soc_floor_barrier=float(pick('w_soc_floor_barrier', 0.0)), w_uncertainty_reserve=float(pick('w_uncertainty_reserve', 0.0)), speed_quartic_scale_kmh=float(pick('speed_quartic_scale_kmh', 80.0)), progress_lag_deadband_kmh=float(pick('progress_lag_deadband_kmh', 0.0)), soc_solar_headroom_max=float(pick('soc_solar_headroom_max', 0.92)), solar_headroom_power_scale_w=float(pick('solar_headroom_power_scale_w', 1000.0)), soc_floor_barrier_eps=float(pick('soc_floor_barrier_eps', 0.01)), reserve_soc_per_hour=float(pick('reserve_soc_per_hour', 0.0)), reserve_soc_max_extra=float(pick('reserve_soc_max_extra', 0.0)), constraint_penalty=float(pick('constraint_penalty', 10000.0))) / default / nested[name] / legacy[name]`
- 主なローカル代入名: `legacy, nested`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `legacy` に `legacy or {}` の結果を代入する。
2. `nested` に `{}` の結果を代入する。
3. 条件 `isinstance(mpc_cfg, dict)` を判定し、真なら内部処理を行う。
4. `nested` に `mpc_cfg.get('upper_cost', {})` or `{}` の結果を代入する。
5. 関数 `pick` を定義する。

6. UpperCostConfig(objective_mode=str(pick('objective_mode', 'weighted')).strip().lower(), w_wait=float(pick('w_wait', 1.0)), w_travel_time=float(pick('w_travel_time', 1.0)), w_terminal_soc_min=float(pick('w_terminal_soc_min', 30.0)), w_day_end_soc_min=float(pick('w_day_end_soc_min', default_drive_window)), w_soc_day_max=float(pick('w_soc_day_max', 10000.0)), w_soc_day_track=float(pick('w_soc_day_track', 0.0)), w_speed_smooth=float(pick('w_speed_smooth', _cfg_value(legacy, 'w_dv', 30.0))), w_dv_limit=float(pick('w_dv_limit', 2.0)), w_speed_limit=float(pick('w_speed_limit', 50.0)), w_drive_window=float(pick('w_drive_window', default_drive_window)), w_current_sq=float(pick('w_current_sq', _cfg_value(legacy, 'w_current', 0.01))), w_pack_energy=float(pick('w_pack_energy', 0.0)), w_joule_loss=float(pick('w_joule_loss', 0.0)), w_aero_energy=float(pick('w_aero_energy', 0.0)), w_mech_energy=float(pick('w_mech_energy', 0.0)), w_speed_quartic=float(pick('w_speed_quartic', 0.0)), w_solar_headroom=float(pick('w_solar_headroom', 0.0)), w_progress_lag=float(pick('w_progress_lag', 0.0)), w_progress_terminal_lag=float(pick('w_progress_terminal_lag', 0.0)), w_kinetic_pos=float(pick('w_kinetic_pos', 0.0)), w_pack_power_slew=float(pick('w_pack_power_slew', 0.0)), w_temp=float(pick('w_temp', _cfg_value(legacy, 'w_T', 5.0))), w_soc_terminal=float(pick('w_soc_terminal', 0.0)), w_soc_floor_barrier=float(pick('w_soc_floor_barrier', 0.0)), w_uncertainty_reserve=float(pick('w_uncertainty_reserve', 0.0)), speed_quartic_scale_kmh=float(pick('speed_quartic_scale_kmh', 80.0)), progress_lag_deadband_kmh=float(pick('progress_lag_deadband_kmh', 0.0)), soc_solar_headroom_max=float(pick('soc_solar_headroom_max', 0.92)), solar_headroom_power_scale_w=float(pick('solar_headroom_power_scale_w', 1000.0)), soc_floor_barrier_eps=float(pick('soc_floor_barrier_eps', 0.01)), reserve_soc_per_hour=float(pick('reserve_soc_per_hour', 0.0)), reserve_soc_max_extra=float(pick('reserve_soc_max_extra', 0.0)), constraint_penalty=float(pick('constraint_penalty', 10000.0))) を返す。

代表コード断片

```
def load_upper_cost_config(
    mpc_cfg: Optional[dict],
    *,
    legacy: Optional[dict] = None,
    default_drive_window: float = 1.0e5,
) -> UpperCostConfig:
    legacy = legacy or {}
    nested = {}
    if isinstance(mpc_cfg, dict):
        nested = mpc_cfg.get("upper_cost", {}) or {}

    def pick(name: str, default):
        if name in nested:
            return nested[name]
        if name in legacy:
            return legacy[name]
        return default

    return UpperCostConfig(
        objective_mode=str(pick("objective_mode", "weighted")).strip().lower(),
        w_wait=float(pick("w_wait", 1.0)),
        w_travel_time=float(pick("w_travel_time", 1.0)),
        w_terminal_soc_min=float(pick("w_terminal_soc_min", 30.0)),
        w_day_end_soc_min=float(pick("w_day_end_soc_min", default_drive_window)),
        w_soc_day_max=float(pick("w_soc_day_max", 1.0e4)),
        w_soc_day_track=float(pick("w_soc_day_track", 0.0)),
        w_speed_smooth=float(pick("w_speed_smooth", _cfg_value(legacy, "w_dv", 30.0))),
        w_dv_limit=float(pick("w_dv_limit", 2.0)),
        w_speed_limit=float(pick("w_speed_limit", 50.0)),
        w_drive_window=float(pick("w_drive_window", default_drive_window)),
        w_current_sq=float(pick("w_current_sq", _cfg_value(legacy, "w_current", 0.01))),
        w_pack_energy=float(pick("w_pack_energy", 0.0)),
        w_joule_loss=float(pick("w_joule_loss", 0.0)),
        w_aero_energy=float(pick("w_aero_energy", 0.0)),
        w_mech_energy=float(pick("w_mech_energy", 0.0)),
        ...
```

7.5 L65 関数 `load_upper_cost_config.pick`

- 行範囲: L65–L70
- 所属: `load_upper_cost_config`
- 定義: `pick(name:str,default)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: `default / nested[name] / legacy[name]`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. 条件 `name in nested` を判定し、真なら内部処理を行う。
2. `nested[name]` を返す。
3. 条件 `name in legacy` を判定し、真なら内部処理を行う。
4. `legacy[name]` を返す。
5. `default` を返す。

代表コード断片

```
def pick(name: str, default):
    if name in nested:
        return nested[name]
    if name in legacy:
        return legacy[name]
    return default
```

7.6 L110 関数 `quad_penalty`

- 行範囲: L110–L115
- 所属: `module` 直下
- 定義: `quad_penalty(x:float,cap:float=1000.0)->float`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: `x * x / 0.0`

- 主なローカル代入名: `x`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出す例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- `'->'`以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `x <= 0.0` を判定し、真なら内部処理を行う。
2. `0.0` を返す。
3. 条件 `x > cap` を判定し、真なら内部処理を行う。
4. `x` に `cap` の結果を代入する。
5. `x * x` を返す。

代表コード断片

```
def quad_penalty(x: float, cap: float = 1.0e3) -> float:
    if x <= 0.0:
        return 0.0
    if x > cap:
        x = cap
    return x * x
```

7.7 L118 関数 `upper_stage_cost`

- 行範囲: L118–L260
- 所属: `module` 直下
- 定義: `upper_stage_cost(cfg:UpperCostConfig,*,dt_wait:float,dt_travel:float,v_kmh:float,v_prev_kmh:float,vmax_local_kmh:float,drive_limits:Optional[tuple],dv_limit_kmhps:float,I_a:float,V_v:float,P_pv_w:float,P_pack_w:float,P_pack_prev_w:Optional[float],P_mech_wheel_w:float,losses_int_w:float,losses_line_w:float,F_aero_n:float,kinetic_step_wh:float,z_next:float,Tb_next_c:float,term_soc_min:float,soc_min:float,soc_max:float,temp_min_c:float,temp_max_c:float,day_end_soc_min:Optional[float],day_end_crossing:bool,soc_day_end_max:float,soc_day_track_target:Optional[float],soc_day_track_tol:float,I_max:float,I_chg_min:float,V_min:float,V_max:float,time_ahead_h:float,progress_lag_km:float=0.0)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `max`, `min`, `quad_penalty`
- 戻り値の要点: `J / J`
- 主なローカル代入名: `J`, `c`, `d_pack_kw`, `dv`, `e_aero_wh`, `e_loss_wh`, `e_mech_wh`, `e_pack_wh`, `lag_err`, `pv_kw`, `reserve_extra`, `reserve_soc`, `soc_gap`, `solar_scale`, `speed_scale`, `vmax_kmh`, `vmin_kmh`

- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 18、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. J に 0.0 の結果を代入する。
2. 条件 `dt_wait > 0.0` を判定し、真なら内部処理を行う。
3. J を Add で更新する。
4. J を Add で更新する。
5. 条件 `cfg.objective_mode in {'fastest', 'fastest_feasible', 'minimum_time'}` を判定し、真なら内部処理を行う。
6. c に `cfg.constraint_penalty` の結果を代入する。
7. dv に `(v_kmh - v_prev_kmh) / max(dt_travel, 0.001)` の結果を代入する。
8. 条件 `dv_limit_kmhps > 0.0` を判定し、真なら内部処理を行う。
9. J を Add で更新する。
10. 条件 `drive_limits is not None` を判定し、真なら内部処理を行う。
11. `(vmin_kmh, vmax_kmh)` に `drive_limits` の結果を代入する。
12. J を Add で更新する。
13. J を Add で更新する。
14. J を Add で更新する。
15. 条件 `day_end_crossing and day_end_soc_min is not None` を判定し、真なら内部処理を行う。
16. J を Add で更新する。
17. 条件 `soc_day_end_max > 0.0` を判定し、真なら内部処理を行う。
18. J を Add で更新する。
19. J を Add で更新する。
20. J を Add で更新する。
21. J を Add で更新する。
22. J を Add で更新する。
23. J を Add で更新する。
24. J を Add で更新する。
25. J を Add で更新する。

26. J を Add で更新する。
27. J を返す。
28. J を Add で更新する。
29. 条件 `soc_day_track_target is not None` を判定し、真なら内部処理を行う。
30. J を Add で更新する。
31. J を Add で更新する。
32. `dv` に $(v_kmh - v_prev_kmh) / \max(dt_travel, 0.001)$ の結果を代入する。
33. J を Add で更新する。
34. 条件 `dv_limit_kmhps > 0.0` を判定し、真なら内部処理を行う。
35. J を Add で更新する。
36. 条件 `drive_limits is not None` を判定し、真なら内部処理を行う。
37. $(vmin_kmh, vmax_kmh)$ に `drive_limits` の結果を代入する。
38. J を Add で更新する。
39. J を Add で更新する。
40. J を Add で更新する。
41. J を Add で更新する。
42. `e_pack_wh` に $\max(0.0, P_pack_w) * dt_travel / 3600.0$ の結果を代入する。
43. `e_loss_wh` に $\max(0.0, losses_int_w + losses_line_w) * dt_travel / 3600.0$ の結果を代入する。
44. `e_aero_wh` に $\max(0.0, F_aero_n) * (v_kmh / 3.6) * dt_travel / 3600.0$ の結果を代入する。
45. `e_mech_wh` に $\max(0.0, P_mech_wheel_w) * dt_travel / 3600.0$ の結果を代入する。
46. J を Add で更新する。
47. J を Add で更新する。
48. J を Add で更新する。
49. J を Add で更新する。
50. J を Add で更新する。
51. 条件 `cfg.w_speed_quartic > 0.0` を判定し、真なら内部処理を行う。
52. `speed_scale` に $\max(1.0, cfg.speed_quartic_scale_kmh)$ の結果を代入する。
53. J を Add で更新する。
54. 条件 `cfg.w_pack_power_slew > 0.0 and P_pack_prev_w is not None` を判定し、真なら内部処理を行う。
55. `d_pack_kw` に $(\text{float}(P_pack_w) - \text{float}(P_pack_prev_w)) / 1000.0$ の結果を代入する。
56. J を Add で更新する。
57. 条件 `cfg.w_progress_lag > 0.0` を判定し、真なら内部処理を行う。
58. `lag_err` に $\max(0.0, \text{float}(\text{progress_lag_km}) - \text{float}(cfg.progress_lag_deadband_km))$ の結果を代入する。
59. J を Add で更新する。

60. 条件 `cfg.w_solar_headroom > 0.0` を判定し、真なら内部処理を行う。
61. `solar_scale` に `max(1.0, cfg.solar_headroom_power_scale_w)` の結果を代入する。
62. `p_v_kw` に `max(0.0, P_p_v_w) / solar_scale` の結果を代入する。
63. 条件 `p_v_kw > 0.0` を判定し、真なら内部処理を行う。
64. `J` を `Add` で更新する。
65. 条件 `cfg.w_soc_floor_barrier > 0.0` を判定し、真なら内部処理を行う。
66. `soc_gap` に `max(float(z_next) - float(soc_min), float(cfg.soc_floor_barrier_eps))` の結果を代入する。
67. `J` を `Add` で更新する。
68. 条件 `cfg.w_uncertainty_reserve > 0.0 and cfg.reserve_soc_per_hour > 0.0` を判定し、真なら内部処理を行う。
69. `reserve_extra` に `min(max(0.0, float(cfg.reserve_soc_max_extra)), max(0.0, float(time_ahead_h)) * max(0.0, float(cfg.reserve_soc_per_hour)))` の結果を代入する。
70. `reserve_soc` に `min(float(soc_max), float(soc_min) + reserve_extra)` の結果を代入する。
71. `J` を `Add` で更新する。
72. 条件 `day_end_crossing and day_end_soc_min is not None` を判定し、真なら内部処理を行う。
73. `J` を `Add` で更新する。
74. 条件 `soc_day_end_max > 0.0` を判定し、真なら内部処理を行う。
75. `J` を `Add` で更新する。
76. `c` に `cfg.constraint_penalty` の結果を代入する。
77. `J` を `Add` で更新する。
78. `J` を `Add` で更新する。
79. `J` を `Add` で更新する。
80. `J` を `Add` で更新する。

代表コード断片

```
def upper_stage_cost(
    cfg: UpperCostConfig,
    *,
    dt_wait: float,
    dt_travel: float,
    v_kmh: float,
    v_prev_kmh: float,
    vmax_local_kmh: float,
    drive_limits: Optional[tuple],
    dv_limit_kmhps: float,
    I_a: float,
    V_v: float,
    P_pv_w: float,
    P_pack_w: float,
    P_pack_prev_w: Optional[float],
    P_mech_wheel_w: float,
    losses_int_w: float,
    losses_line_w: float,
    F_aero_n: float,
    kinetic_step_wh: float,
```

```

z_next: float,
Tb_next_c: float,
term_soc_min: float,
soc_min: float,
soc_max: float,
temp_min_c: float,
temp_max_c: float,
day_end_soc_min: Optional[float],
day_end_crossing: bool,
soc_day_end_max: float,
soc_day_track_target: Optional[float],
soc_day_track_tol: float,
I_max: float,
I_chg_min: float,
V_min: float,
...

```

7.8 L263 関数 upper_terminal_cost

- 行範囲: L263–L280
- 所属: module 直下
- 定義: `upper_terminal_cost(cfg:UpperCostConfig,*,z_terminal:float,term_soc_min:float,soc_finish_target:float,soc_finish_tol:float=0.0,progress_terminal_lag_km:float=0.0)->float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float, max, quad_penalty`
- 戻り値の要点: J
- 主なローカル代入名: J, lag_err
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. J に `cfg.constraint_penalty * quad_penalty(term_soc_min - z_terminal)` の結果を代入する。
2. 条件 `soc_finish_target > 0.0` を判定し、真なら内部処理を行う。
3. J を Add で更新する。
4. 条件 `cfg.w_progress_terminal_lag > 0.0` を判定し、真なら内部処理を行う。
5. lag_err に `max(0.0, float(progress_terminal_lag_km) - float(cfg.progress_lag_deadband_km))` の結果を代入する。
6. J を Add で更新する。
7. J を返す。

代表コード断片

```
def upper_terminal_cost(
    cfg: UpperCostConfig,
    *,
    z_terminal: float,
    term_soc_min: float,
    soc_finish_target: float,
    soc_finish_tol: float = 0.0,
    progress_terminal_lag_km: float = 0.0,
) -> float:
    J = cfg.constraint_penalty * quad_penalty(term_soc_min - z_terminal)
    if soc_finish_target > 0.0:
        J += cfg.w_soc_terminal * quad_penalty(
            z_terminal - (soc_finish_target + max(0.0, soc_finish_tol))
        )
    if cfg.w_progress_terminal_lag > 0.0:
        lag_err = max(0.0, float(progress_terminal_lag_km) - float(cfg.
            progress_lag_deadband_km))
        J += cfg.w_progress_terminal_lag * quad_penalty(lag_err)
    return J
```

7.9 L283 関数 active_upper_cost_terms

- 行範囲: L283–L288
- 所属: module 直下
- 定義: `active_upper_cost_terms(cfg:UpperCostConfig,threshold:float=1e-06)->Dict[str, float]`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `abs`, `float`, `items`, `startswith`, `to_dict`
- 戻り値の要点: `out`
- 主なローカル代入名: `key`, `out`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 1、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `out` に `{}` の結果を代入する。
2. `cfg.to_dict().items()` を順に走査し、各要素を `(key, value)` に入れて処理する。
3. 条件 `key.startswith('w_')` and `abs(float(value)) > threshold` を判定し、真なら内部処理を行う。
4. `out[key]` に `float(value)` の結果を代入する。
5. `out` を返す。

代表コード断片

```
def active_upper_cost_terms(cfg: UpperCostConfig, threshold: float = 1.0e-6) -> Dict[str, float]:
    out = {}
    for key, value in cfg.to_dict().items():
        if key.startswith("w_") and abs(float(value)) > threshold:
            out[key] = float(value)
    return out
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/upper_cost.py` を **planner helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。